Digital System Design with Verilog Course 03603473-65	Lab 7 FSM

Lab 7 —Finite State Machine (FSM)

Name: Verilog Student	Student ID: 64010000	Date: Week 7	Points: 100/100

ถูก/ผิด	แบบฝึกหัดข้อ 1 — Moore vs Mealy: ระบุถูกหรือผิด	10 pts/_____

อ่านคำกล่าวแต่ละข้อ แล้วเขียน ถูก หรือ ผิด ในช่องว่าง ถ้าผิดให้เขียนเหตุผลสั้นๆ:

ข้อ	คำกล่าว	ถูก/ผิด	เหตุผล (ถ้าผิด)
1	Moore FSM มี output ที่ขึ้นกับ state เพียงอย่างเดียว ไม่ว่า input จะเปลี่ยนแปลงอย่างไร output ก็ไม่เปลี่ยนจนกว่า state จะเปลี่ยน	ถูก	
2	Mealy FSM ต้องเขียนด้วยรูปแบบ 3-always-block เหมือน Moore FSM เสมอ	ผิด	Mealy มักใช้ 2-always block (รวม next-state + output)
3	ถ้า case statement ใน combinational always block ไม่ครอบคลุมทุกกรณี (case) และไม่มี default จะทำให้ synthesis tool สร้าง latch ขึ้นมา	ถูก	
4	ใน state diagram ของ Moore FSM ค่า output จะเขียนไว้บนลูกศร (transition) ระหว่าง state	ผิด	Moore เขียน output ในวงกลม state (บนลูกศรคือ Mealy)
5	Mealy FSM สามารถสร้าง output ตอบสนองได้เร็วกว่า Moore FSM อยู่ 1 clock cycle เพราะไม่ต้องรอ state เปลี่ยนก่อน	ถูก	
6	One-hot encoding ใช้จำนวน flip-flop น้อยกว่า binary encoding เสมอ จึงประหยัด area มากกว่า	ผิด	One-hot ใช้ N FFs มากกว่า binary ที่ใช้ ceil(log2 N) FFs
7	always @(*) ใน Verilog หมายถึง combinational logic ที่ sensitive ต่อทุกสัญญาณที่ถูกอ่านภายในบล็อก	ถูก	
8	ควรใช้ non-blocking assignment (<=) ใน combinational always block เพื่อป้องกัน race condition	ผิด	combinational ต้องใช้ blocking (=) เพื่อหลีกเลี่ยง race
9	FSM ที่มี 5 สถานะ ต้องใช้ state register อย่างน้อย 3 บิต เมื่อใช้ binary encoding	ถูก	
10	ถ้าใช้ CLOCK_50 (50 MHz) เป็นสัญญาณนาฬิกาให้ sequence detector ที่ป้อน input ด้วยสวิตช์บนบอร์ด จะทำงานได้ปกติโดยไม่ต้องทำ edge detection	ผิด	ปุ่มกดนานหลาย ms ที่ 50MHz จะทริกเกอร์ซ้ำนับล้านรอบ

เติมช่องว่าง	แบบฝึกหัดข้อ 2 — ความรู้พื้นฐาน FSM	10 pts/_____

เติมคำหรือตัวเลขที่ถูกต้องลงในช่องว่าง:

1. โครงสร้างโค้ดของ Moore FSM ใช้ 3 always block แบ่งเป็น state register, next-state logic, และ output logic
2. โครงสร้างโค้ดของ Mealy FSM ใช้ 2 always block โดยรวม next-state logic กับ output logic ไว้ในบล็อกเดียวกัน
3. Output ของ Moore FSM เป็นฟังก์ชันของ state ปัจจุบัน เพียงอย่างเดียว
4. Output ของ Mealy FSM เป็นฟังก์ชันของ state ปัจจุบัน และ input ปัจจุบัน
5. ถ้า case statement ไม่ครบทุก case ใน combinational always block จะเกิดปัญหาที่เรียกว่า Latch ซึ่งเป็นสิ่งที่ไม่พึงประสงค์ใน synchronous design
6. การหาร CLOCK_50 (50 MHz) ให้ได้ความถี่ 1 Hz ต้องนับจาก 0 ถึง 49_999_999
7. ปุ่ม KEY บนบอร์ด DE0 เป็นแบบ active กล่าวคือ ไม่กด = 1, กด = 0
8. ใน sequential always block ควรใช้ non-blocking assignment (เครื่องหมาย <= เพื่อป้องกัน race condition
9. Overlapping sequence detection คือการตรวจจับที่อนุญาตให้ลำดับที่ตรวจพบแล้ว reuse บิตท้ายกับลำดับถัดไปได้
10. FSM ที่มี 6 สถานะ ใช้ binary encoding ต้องการ state register ขนาด 3 บิต แต่ถ้าใช้ one-hot encoding ต้องการ 6 บิต

จับคู่	แบบฝึกหัดข้อ 3 — จับคู่คำศัพท์ FSM	8 pts/_____

จับคู่คำศัพท์/แนวคิดในคอลัมน์ซ้ายกับคำอธิบายในคอลัมน์ขวา เขียนตัวเลขคำตอบลงในช่อง "คู่กับ":

	คำศัพท์ / แนวคิด	คู่กับข้อ
A	State Register	2
B	Next-State Logic (combinational)	8
C	Output Logic ของ Moore FSM	4
D	Output Logic ของ Mealy FSM	1
E	Inferred Latch	3
F	Edge Detector	7
G	Tick Generator	5
H	One-hot Encoding	6

ข้อ	คำอธิบาย
1	f(state, input) — output ขึ้นกับทั้ง state และ input ร่วมกัน
2	always @(posedge clk or negedge rst_n) — sequential logic ที่เก็บค่า state
3	เกิดจาก case/if ที่ไม่ครบทุกกรณีใน combinational always block
4	f(state) — output ขึ้นกับ state เพียงอย่างเดียว ไม่มี input เกี่ยวข้อง
5	แปลง clock ความถี่สูง (เช่น 50 MHz) เป็น pulse ความถี่ต่ำ (เช่น 1 Hz)
6	ใช้ 1 flip-flop ต่อ 1 state — FSM 6 สถานะ ใช้ register 6 บิต
7	ตรวจจับขอบขึ้น/ลงของสัญญาณ แล้วสร้าง pulse กว้าง 1 คลอก
8	always @(*) case(state) ... next_state = ... — คำนวณ state ถัดไป

DEBUG	แบบฝึกหัดข้อ 4-7 — หาบัคในโค้ด (4 ข้อ ข้อละ 5 คะแนน)	20 pts/_____

โค้ดแต่ละข้อมีบัค 1 จุด ให้ระบุบรรทัดที่ผิด อธิบายว่าผิดอย่างไร และเขียนโค้ดที่ถูกต้อง:

VERILOG · โค้ดที่ 1 (มีบัค 1 จุด)

```
1. // Moore FSM output logic — traffic light
2. always @(*)
3. case (state)
4.    RED:    LEDG = 3'b001;
5.    GREEN:  LEDG = 3'b100;
6.    // YELLOW ????
7.  endcase
```

บัคอยู่ตรงไหน: บรรทัดที่ 3–7 (ขาดกรณี YELLOW และไม่มี default)

ผิดอย่างไร / ผลกระทบ: case ไม่ครอบคลุมทุกสถานะ ทำให้เกิด Inferred Latch ซึ่งไม่พึงประสงค์

แก้ไขอย่างไร: เพิ่ม YELLOW: LEDG = 3'b010; และเพิ่ม default: LEDG = 3'b001; ก่อน endcase

VERILOG · โค้ดที่ 2 (มีบัค 1 จุด)

```
1. // Moore FSM — output logic for door lock
2. always @(*) begin
3.  case (state)
4.     IDLE:    unlocked = 1'b0;
5.     VERIFY:  unlocked = (key_match) ? 1'b1 : 1'b0;
6.     OPEN:    unlocked = 1'b1;
7.     default: unlocked = 1'b0;
8.  endcase
```

บัคอยู่ตรงไหน: บรรทัดที่ 5 (VERIFY: unlocked = (key_match) ? 1'b1 : 1'b0;)

ผิดอย่างไร / ผลกระทบ: ผิดกฎ Moore FSM เพราะนำ input (key_match) มาใช้ใน output logic ทำให้เกิด glitch

แก้ไขอย่างไร: ตัด key_match ออก ให้ VERIFY: unlocked = 1'b0; แล้วนำ key_match ไปเช็คใน next-state logic

VERILOG · โค้ดที่ 4 (มีบัค 1 จุด)

```
1. // Mealy FSM — next-state + output logic
2. always @(state)  // <--- สังเกตบรรทัดนี้
3. begin
4.  detected = 1'b0;
5.  case (state)
6.     S0: next_state = in ? S1 : S0;
7.     S1: next_state = in ? S1 : S2;
8.     S2: next_state = in ? S3 : S0;
9.     S3: begin
10.          next_state = in ? S1 : S2;
11.          detected   = in;  // Mealy output
12.    end
13.  endcase
```

บัคอยู่ตรงไหน: บรรทัดที่ 2 (always @(state))

ผิดอย่างไร / ผลกระทบ: Sensitivity list ขาดสัญญาณ in ทำให้เมื่อ in เปลี่ยนแปลง วงจรจะไม่ประเมินผลใหม่

แก้ไขอย่างไร: แก้ไขเป็น always @(*) เพื่อให้ครอบคลุมสัญญาณ input ทั้งหมดที่ถูกอ่านในบล็อก

DESIGN	แบบฝึกหัดข้อ 8 — วิเคราะห์สเปกและออกแบบ State Diagram	0 pts/_____

8.1 สถานการณ์จำลอง (Scenario)

ออกแบบวงจรล็อครหัสดิจิทัลที่ใช้รหัสผ่าน 3 หลัก (เลข 0-9 ต่อหลัก) เพื่อควบคุมการปลดล็อค ระบบต้องมีกลไกป้องกันการเดารหัส (brute-force protection) โดยล็อคชั่วคราวหลังกดรหัสผิดติดต่อกัน 3 ครั้ง และมีระบบล็อคอัตโนมัติ (auto-lock) หลังปลดล็อคสำเร็จแล้วไม่ได้ใช้งานภายใน 5 วินาที

8.2 สเปกของระบบ

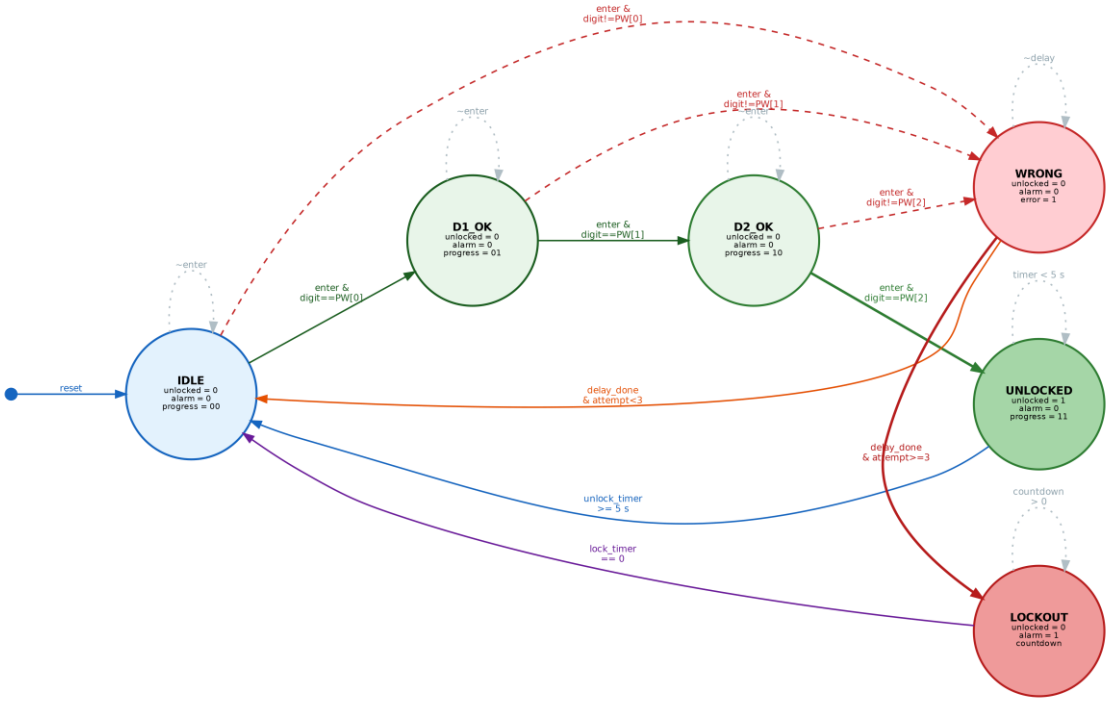
หัวข้อ	รายละเอียด
รหัสผ่าน	3 หลัก: 3-7-1 (hard-coded ใน parameter ของโมดูล)
การป้อนรหัส	ตั้งค่าตัวเลข (0-9) ที่ SW[3:0] แล้วกดปุ่ม KEY[1] เพื่อยืนยันแต่ละหลัก
ป้อนถูก 3 หลัก	ปลดล็อค (LEDG[0] ติด) · progress LED แสดงจำนวนหลักที่ถูก
ป้อนผิด	แสดงข้อผิดพลาด 1 วินาที แล้วกลับไปเริ่มต้นใหม่ · เพิ่ม attempt counter
ผิดติดกัน 3 ครั้ง	ล็อคชั่วคราว (LOCKOUT) 10 วินาที · LEDR ทุกดวงติด · นับถอยหลังบน HEX0
Auto-lock	หลังปลดล็อคสำเร็จ ถ้าไม่กดปุ่มภายใน 5 วินาที ระบบล็อคกลับอัตโนมัติ
Reset	KEY[0] (active-low) กลับสู่สถานะเริ่มต้น ล้าง counter ทั้งหมด

8.3 วิเคราะห์ Input / Output

สัญญาณ	ทิศทาง	ขนาด	คำอธิบาย
CLOCK_50	Input	1 bit	สัญญาณนาฬิกา 50 MHz จากบอร์ด
KEY[0]	Input	1 bit	Reset (active-low)
KEY[1]	Input	1 bit	ปุ่มยืนยัน (enter) — ต้อง edge-detect
SW[3:0]	Input	4 bit	ตัวเลขที่ป้อน (0-9)
LEDG[0]	Output	1 bit	unlocked — ติดเมื่อปลดล็อคสำเร็จ
LEDG[2:1]	Output	2 bit	progress — จำนวนหลักที่ป้อนถูก
LEDR[9:0]	Output	10 bit	alarm — ติดทุกดวงระหว่าง LOCKOUT
HEX0[6:0]	Output	7 bit	attempt count (ปกติ) หรือ countdown (LOCKOUT)

TIP: Output ทุกตัวของ FSM ขึ้นกับ state เพียงอย่างเดียว (ไม่มี input ปน) — นี่คือลักษณะเฉพาะของ Moore FSM

8.4 State Diagram



เส้นทึบเขียว = ป้อนรหัสถูก · เส้นประแดง = ป้อนรหัสผิด · เส้นน้ำเงิน = timeout · เส้นม่วง = lockout หมดเวลา

8.5 State Table (ตารางสถานะ)

Current State	เงื่อนไข	Next State	หมายเหตุ
IDLE	enter & digit==PW[0]	D1_OK	หลักแรกถูก
IDLE	enter & digit!=PW[0]	WRONG	หลักแรกผิด
IDLE	~enter	IDLE	รอต่อ (self-loop)
D1_OK	enter & digit==PW[1]	D2_OK	หลักที่ 2 ถูก
D1_OK	enter & digit!=PW[1]	WRONG	หลักที่ 2 ผิด
D2_OK	enter & digit==PW[2]	UNLOCKED	รหัสครบถูกหมด
D2_OK	enter & digit!=PW[2]	WRONG	หลักที่ 3 ผิด
UNLOCKED	unlock_timer >= 5s	IDLE	Auto-lock timeout
WRONG	delay_done & attempt < 3	IDLE	ยังลองได้อีก
WRONG	delay_done & attempt >= 3	LOCKOUT	ผิดครบ 3 ครั้ง
LOCKOUT	lock_timer == 0	IDLE	หมดเวลาล็อค

8.6 State Encoding

State	รหัส (3 บิต)	unlocked	alarm (LEDR)	progress	error
IDLE	3'b000	0	0	00	0
D1_OK	3'b001	0	0	01	0
D2_OK	3'b010	0	0	10	0
UNLOCKED	3'b011	1	0	11	0

State	รหัส (3 บิต)	unlocked	alarm (LEDR)	progress	error
WRONG	3'b100	0	0	00	1
LOCKOUT	3'b101	0	1	00	0

8.7 โครงสร้าง Sub-Module

ไฟล์	Sub-Module	หน้าที่	สถานะ
combo_lock_top.v	Top module	เชื่อมต่อทุก sub-module เข้าด้วยกัน	ให้มาครบ
edge_detector.v	Key Edge Detector	ตรวจจับ falling edge ของ KEY[1]	ต้องเติม
tick_gen.v	1 Hz Tick Generator	หาร 50 MHz ⟶ 1 pulse/วินาที	ต้องเติม
combo_lock_fsm.v	FSM Core	State reg + Next-state + Output + Timers	ต้องเติม
hex_decode.v	7-Segment Decoder	แปลงเลข 4 บิต ⟶ HEX display	จาก Lab 6

เพิ่มไฟล์เหล่านี้เข้า Quartus project ตามที่เป็น · ไม่ต้องแก้ไขอะไร

combo_lock_top.v — Top Module (COMPLETE)

```
module combo_lock_top (
    input         CLOCK_50,
    input  [1:0]  KEY,        // KEY[0]=rst_n, KEY[1]=enter
    input  [3:0]  SW,         // digit input (0-9)
    output [2:0]  LEDG,       // {progress[2:1], unlocked}
    output [9:0]  LEDR,       // alarm
    output [6:0]  HEX0        // 7-seg display
);

    wire rst_n = KEY[0];
    wire enter, tick, unlocked;
    wire [1:0] progress;
    wire [9:0] alarm;
    wire [1:0] attempt_cnt;
    wire [3:0] lock_timer;

    // Sub-module instantiations
    edge_detector u_edge (
        .clk(CLOCK_50), .rst_n(rst_n),
        .key_in(KEY[1]), .press(enter) );

    tick_gen u_tick (
        .clk(CLOCK_50), .rst_n(rst_n), .tick(tick) );

    combo_lock_fsm u_fsm (
        .clk(CLOCK_50), .rst_n(rst_n),
        .enter(enter), .digit(SW), .tick(tick),
        .unlocked(unlocked), .progress(progress),
        .alarm(alarm), .attempt_out(attempt_cnt),
        .lock_timer_out(lock_timer) );

    // Display select: LOCKOUT -> countdown, else -> attempt count
    wire [3:0] hex_val = (alarm != 10'd0) ?
                         lock_timer : {2'b00, attempt_cnt};
    hex_decode u_hex (.value(hex_val), .seg(HEX0));
```

```
    assign LEDG = {progress, unlocked};
    assign LEDR = alarm;
endmodule
```

```
ั
```

COMPLETE **แบบฝึกหัดข้อ 9 — เขียน edge_detector.v** 5 pts/_____

หลักการทำงาน: ใช้ 2-FF shift register จับค่า key_in 2 คลอกติดกัน ถ้า r2=1 (คลอกก่อน) และ r1=0 (คลอกนี้) แปลว่ามี falling edge (ปุ่ม active-low ถูกกด)

ทำไมต้องทำ: ปุ่มกดด้วยมือค้างอยู่หลาย ms = หลายล้าน clock cycles ที่ 50 MHz ถ้าไม่ detect edge ระบบจะ enter ซ้ำนับล้านครั้ง

Port	Direction	Width	Behavior
clk	input	1 bit	CLOCK_50 (50 MHz)
rst_n	input	1 bit	Active-low async reset
key_in	input	1 bit	Active-low push button (KEY[1])
press	output	1 bit	1-clock pulse เมื่อตรวจจับ falling edge

edge_detector.v — เติมช่องว่าง 5 ช่อง

```
module edge_detector (
    input  wire  clk,
    input  wire  rst_n,   // active-low reset
    input  wire  key_in,  // active-low button
    output wire  press    // 1-clock pulse on button press
);
    reg r1, r2;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            r1 <=  1'b1;          // 1. reset value? (ปุ่มไม่กด = ?)
            r2 <=  1'b1;          // 2. reset value?
        end
        else begin
            r1 <=  key_in;        // 3. sample key_in เข้า FF ตัวแรก
            r2 <=  r1;            // 4. delay r1 อีก 1 คลอก
        end
    assign press =  r2 & ~r1;     // 5. falling edge: r2 เดิมสูง, r1 ใหม่ต่ำ
endmodule
```

COMPLETE **แบบฝึกหัดข้อ 10 — เขียน tick_gen.v** 5 pts/_____

หลักการทำงาน: นับ clock cycle จาก 0 ถึง 49,999,999 (50 ล้าน-1) แล้ว reset กลับ 0 ทำให้ได้ pulse 1 ครั้งต่อวินาที

ขนาด counter: ต้องใช้ 26 บิต เพราะ $\log_2(50{,}000{,}000) \approx 25.6$

Port	Direction	Width	Behavior
clk	input	1 bit	CLOCK_50 (50 MHz)
rst_n	input	1 bit	Active-low async reset
tick	output	1 bit	1-clock pulse ทุก 1 วินาที

tick_gen.v — เติมช่องว่าง 4 ช่อง

```
module tick_gen (
    input  wire  clk,
    input  wire  rst_n,
    output wire  tick     // 1 pulse per second
);
    reg [25:0] cnt;
    assign tick = (cnt == 26'd49_999_999); // 1. terminal count สำหรับ 1 Hz?
    always @(posedge clk or negedge rst_n)
        if (!rst_n)    cnt <=  26'd0;        // 2. reset
        else if (tick) cnt <=  26'd0;        // 3. wrap around
        else           cnt <=  cnt + 1'b1; // 4. increment
endmodule
```

COMPLETE | **แบบฝึกหัดข้อ 11 — เขียน combo_lock_fsm.v (FSM Core)** | 25 pts/_____

Port	Direction	Width	Behavior
clk	input	1 bit	CLOCK_50 (50 MHz)
rst_n	input	1 bit	Active-low async reset
enter	input	1 bit	1-clock pulse จาก edge_detector
digit	input	4 bit	ตัวเลขที่ป้อน (SW[3:0])
tick	input	1 bit	1-clock pulse ทุก 1 วินาที
unlocked	output	1 bit	1 เมื่อ state == UNLOCKED
progress	output	2 bit	00=ยังไม่ได้, 01=1 หลัก, 10=2 หลัก, 11=ครบ
alarm	output	10 bit	10'h3FF เมื่อ LOCKOUT, 0 อื่นๆ
attempt_out	output	2 bit	จำนวนครั้งที่ผิด (0-3)
lock_timer_out	output	4 bit	ค่า countdown ของ LOCKOUT (10⟶0)

⚠ ห้ามใส่ SW, enter, digit หรือ input ใดๆ ใน output logic (TODO 3) — ถ้ามี input ปนจะกลายเป็น Mealy FSM ทันที

combo_lock_fsm.v — เติมส่วน TODO 1-4

```
module combo_lock_fsm (
    input  wire       clk, rst_n, enter, tick,
    input  wire [3:0] digit,
    output reg        unlocked,
    output reg  [1:0] progress,
    output reg  [9:0] alarm,
    output wire [1:0] attempt_out,
    output wire [3:0] lock_timer_out
);

  // ─── Password (ปรับเปลี่ยนได้) ───
  localparam [3:0] PW0=4'd3, PW1=4'd7, PW2=4'd1;

  // ─── State encoding ───
  localparam [2:0]
    IDLE=3'b000, D1_OK=3'b001, D2_OK=3'b010,
    UNLOCKED=3'b011, WRONG=3'b100, LOCKOUT=3'b101;

  reg [2:0] state, next_state;
  reg [1:0] attempt_cnt;  reg [2:0] unlock_timer;
  reg [3:0] lock_timer;   reg       wrong_timer;
  wire      delay_done = wrong_timer;

  // ═══════ TODO 1: State Register ═══════
  always @(posedge clk or negedge rst_n)
    if (!rst_n)  state <=  IDLE;
    else         state <=  next_state;

  // ═══════ TODO 2: Next-State Logic (6 states) ═══════
  always @(*) begin
    next_state = state;  // default: คงสถานะเดิม
    case (state)
      IDLE:     if (enter) next_state = (digit==PW0) ? D1_OK : WRONG;
      D1_OK:    if (enter) next_state = (digit==PW1) ? D2_OK : WRONG;
      D2_OK:    if (enter) next_state = (digit==PW2) ? UNLOCKED : WRONG;
      UNLOCKED: if (unlock_timer >= 3'd5)  next_state = IDLE;
      WRONG:    if (delay_done)
                  next_state = (attempt_cnt>=2'd3) ? LOCKOUT : IDLE;
      LOCKOUT:  if (lock_timer == 4'd0)  next_state = IDLE;
      default:  next_state = IDLE;
    endcase
  end

  // ═══════ TODO 3: Output Logic (Moore — state only!) ═══════
  always @(*) begin
    unlocked=1'b0; progress=2'b00; alarm=10'd0;
    case (state)
      IDLE:     begin unlocked=1'b0; progress=2'b00; alarm=10'd0; end
      D1_OK:    begin unlocked=1'b0; progress=2'b01; alarm=10'd0; end
      D2_OK:    begin unlocked=1'b0; progress=2'b10; alarm=10'd0; end
      UNLOCKED: begin unlocked=1'b1; progress=2'b11; alarm=10'd0; end
      WRONG:    begin unlocked=1'b0; progress=2'b00; alarm=10'd0; end
      LOCKOUT:  begin unlocked=1'b0; progress=2'b00; alarm=10'h3FF; end
    endcase
  end

  // ═══════ TODO 4: Counters & Timers ═══════
  // 4a) Attempt counter: +1 เมื่อเข้า WRONG, reset เมื่อ UNLOCKED/LOCKOUT
```

```
  always @(posedge clk or negedge rst_n)
    if (!rst_n)                          attempt_cnt <= 2'd0;
    else if (state!=WRONG && next_state==WRONG) attempt_cnt <= attempt_cnt + 1'b1;
    else if  (state == UNLOCKED)         attempt_cnt <= 2'd0;
    else if  (state == LOCKOUT)          attempt_cnt <= 2'd0;

  // 4b) WRONG 1-second delay
  always @(posedge clk or negedge rst_n)
    if (!rst_n)           wrong_timer <= 0;
    else if (state != WRONG)  wrong_timer <= 1'b0;
    else if (tick)        wrong_timer <= 1'b1;

  // 4c) UNLOCKED auto-lock timer (นับขึ้น ถึง 5 แล้วกลับ IDLE)
  always @(posedge clk or negedge rst_n)
    if (!rst_n)             unlock_timer <= 0;
    else if (state != UNLOCKED)   unlock_timer <= 3'd0;
    else if (tick)          unlock_timer <= unlock_timer + 1'b1;

  // 4d) LOCKOUT countdown timer (เริ่ม 10 นับลง ถึง 0 แล้วกลับ IDLE)
  always @(posedge clk or negedge rst_n)
    if (!rst_n) lock_timer <= 0;
    else if (state!=LOCKOUT && next_state==LOCKOUT)
      lock_timer <= 4'd10;          // เริ่มนับถอยหลัง
    else if (state==LOCKOUT && tick && lock_timer>0)
      lock_timer <= lock_timer - 1'b1;  // ลดทีละ 1

  assign attempt_out    = attempt_cnt;
  assign lock_timer_out = lock_timer;
endmodule
```

PREDICT	แบบฝึกหัดข้อ 12 — ทำนายผลลัพธ์ (ไม่ต้องจำลอง ใช้ตารางตอบ)	12 pts/_____

12A. ทำนาย Output จาก State (Moore Property)

เนื่องจากเป็น Moore FSM output ขึ้นกับ state เพียงอย่างเดียว ไม่ว่า input จะเป็นอะไร output ก็คงที่ตาม state — ให้เติมค่า output ที่ถูกต้องของแต่ละ state

State	unlocked (LEDG[0])	progress (LEDG[2:1])	alarm (LEDR[9:0])
IDLE	0	00	10'd0
D1_OK	0	01	10'd0
D2_OK	0	10	10'd0
UNLOCKED	1	11	10'd0
WRONG	0	00	10'd0
LOCKOUT	0	00	10'h3FF (ติดหมด)

12B. ทำนาย State Transition จาก State + Input

กำหนด current state และเงื่อนไข ให้เติม next state ที่ถูกต้อง:

#	Current State	เงื่อนไข (Input / Event)	Next State
1	IDLE	enter=1, digit=3 (ถูก)	D1_OK
2	IDLE	enter=1, digit=5 (ผิด)	WRONG
3	IDLE	enter=0 (ไม่กดปุ่ม)	IDLE
4	D1_OK	enter=1, digit=7 (ถูก)	D2_OK
5	D1_OK	enter=1, digit=9 (ผิด)	WRONG
6	D2_OK	enter=1, digit=1 (ถูก)	UNLOCKED
7	D2_OK	enter=1, digit=8 (ผิด)	WRONG
8	UNLOCKED	unlock_timer >= 5	IDLE
9	WRONG	delay_done=1, attempt=1	IDLE
10	WRONG	delay_done=1, attempt=3	LOCKOUT
11	LOCKOUT	lock_timer == 0	IDLE
12	LOCKOUT	lock_timer == 5 (ยังนับอยู่)	LOCKOUT

Board Verification Checklist

ทดสอบแต่ละรายการบนบอร์ดจริง เขียนผลลัพธ์ที่ได้ในช่อง "ผลจริง" แล้วเปรียบเทียบว่าตรงกับที่คาดหวังหรือไม่:

#	ขั้นตอนทดสอบ	ผลที่คาดหวัง	ผลจริง	ผ่าน?
1	กด KEY[0] (reset)	IDLE: LEDG=000, LEDR=off, HEX0=0	IDLE: LEDG=000, LEDR=off, HEX0=0	✓
2	ตั้ง SW=3, กด KEY[1]	D1_OK: LEDG=010 (progress=01)	D1_OK: LEDG=010 (progress=01)	✓

#	ขั้นตอนทดสอบ	ผลที่คาดหวัง	ผลจริง	ผ่าน?
3	ตั้ง SW=7, กด KEY[1]	D2_OK: LEDG=100 (progress=10)	D2_OK: LEDG=100 (progress=10)	✅
4	ตั้ง SW=1, กด KEY[1]	UNLOCKED: LEDG=111 (progress=11, unlocked=1)	UNLOCKED: LEDG=111, unlocked=1	✅
5	รอ 5 วินาทีไม่กดอะไร	Auto-lock: กลับ IDLE, LEDG=000	Auto-lock: กลับ IDLE, LEDG=000	✅
6	ตั้ง SW=5, กด KEY[1] (ผิด)	WRONG ⟶ 1s ⟶ IDLE, HEX0=1	WRONG -> 1s -> IDLE, HEX0=1	✅
7	ป้อนผิดอีก 2 ครั้ง (รวม 3)	LOCKOUT: LEDR ทุกดวงติด, HEX0 นับ 10⟶0	LOCKOUT: LEDR ติดหมด, HEX0=10..0	✅
8	รอ 10 วินาทีใน LOCKOUT	กลับ IDLE: LEDR ดับ, HEX0=0	กลับ IDLE: LEDR ดับ, HEX0=0	✅
9	หลัง lockout ป้อน 3-7-1	UNLOCKED ได้ปกติ (attempt=0)	UNLOCKED ได้ปกติ (attempt=0)	✅
10	ป้อนถูกครึ่งทาง แล้วผิด	D1_OK ⟶ WRONG ⟶ IDLE (เริ่มใหม่)	D1_OK -> WRONG -> IDLE (เริ่มใหม่)	✅